SUMMARY: CONTROL TRANSFER STATEMENTS

SESSION OVERVIEW:

In this session, the students will be able to:

- Understand what control transfer statements are
- Understand the break statement
- Understand the continue statement
- Understand the pass statement

KEY POINTS AND EXAMPLES:

Understand what control transfer statements are

Control transfer statements, also known as control flow statements, are essential components of programming languages like Python. They enable programmers to manage the flow of execution within loops or conditional statements.

In Python, there are three main control transfer statements: break, continue, and pass.

Without control transfer statements:
Without control transfer statements such as break, continue, and pass, programming tasks would be less flexible and more cumbersome. For example, if you wanted to process a list of numbers and stop when you encounter a specific condition, you would have to manually check the condition after each step and decide whether to continue or stop the process. This manual checking would increase the complexity of the code and make it harder to manage.
.
With control transfer statements:
Control transfer statements in Python, such as break, continue, and pass, provide convenient ways to control the flow of execution within loops and conditional statements. They allow you to efficiently handle situations where you need to prematurely exit a loop (break), skip the current iteration and proceed to the next one (continue), or maintain syntactic structure without taking any action (pass). Using these control transfer statements makes your code more concise, readable, and maintainable, as you can express your intentions more clearly and handle various scenarios with ease.

Understand the break statement

Imagine you're in a mall, and you've decided to visit every store on a particular floor. You start walking from one store to the next, checking out the merchandise. However, you've also

set a rule for yourself: if you find the perfect gift for your friend, you'll stop your shopping trip and head straight to the exit.

In this, the act of visiting each store represents an iteration of a loop (e.g., a for or while loop). The loop keeps running until you find the perfect gift or until you've visited all the stores on the floor.

The break statement in Python is like finding that perfect gift. When you encounter a specific condition (e.g., finding the desired item), the break statement kicks in. It's like saying, "Aha! I found what I was looking for! No need to visit the remaining stores."

So, when you execute the break statement in your code, it's similar to finding that perfect gift in the mall. It immediately terminates the loop, and the control flow jumps out of the loop to the next statement after the loop.

Syntax:

```
while condition:
    # Code block
    if condition_to_break:
        break
    # More code
```

or

```
for element in iterable:
    # Code block
    if condition_to_break:
        break
    # More code
```

1. **Usage in Loops:** The break statement is primarily used within loops, such as while loops or for loops. When encountered inside a loop, the break statement immediately exits the loop, regardless of whether the loop's condition has been fully satisfied or not(meaning the loop hasn't iterated through all elements or the condition hasn't evaluated to False yet), the loop immediately exits. It doesn't wait for the loop's natural termination based on the condition.

2. **Condition-Based Termination:** The decision to execute a break statement is often based on a specific condition. Once this condition is met, the loop is terminated, and the program continues with the statement immediately following the loop.

```
for i in range(5):
    print(i)
    if i == 2:
        break
```

The output will be:

```
0
1
2
```

This example demonstrates the usage of break within a for loop. It iterates over the numbers 0 to 4 using range(5). Within each iteration, it prints the current value of i. When i reaches 2, the condition if i == 2: is met, and the break statement is executed, causing an immediate exit from the loop.

Example of using the break statement to stop an infinite loop

```
count = 0
while True:
    print("Inside the loop")
    count += 1
    if count == 3:
        break  # Exit the loop after 3 iterations
```

- Initializes a variable count to 0. This variable will be used to track the number of iterations of the loop.
- Starts an infinite loop. The True condition ensures that the loop will continue indefinitely until a break statement is encountered.
- Prints the message "Inside the loop" to the console during each iteration of the loop.
- Increments the value of count by 1 during each iteration of the loop. This effectively counts the number of iterations.
- Checks if the value of count is equal to 3.
- If the condition count == 3 is met (i.e., after three iterations), the break statement is executed. This causes the loop to exit immediately.

The output will be:

```
Inside the loop
Inside the loop
Inside the loop
```

```
while True:
    user_input = input("Enter 'quit' to exit: ")
    if user_input == 'quit':
        break
    print("You entered:", user_input)
print("Loop exited.")
```

In this example,

- We have a while loop with the condition True, creating an infinite loop.
- Inside the loop, we prompt the user to input a value.
- We check if the user input is equal to 'quit'. If it is, we execute the break statement, which exits the loop immediately.
- If the user input is not 'quit', we print the input value.
- After exiting the loop, we print "Loop exited." to indicate that the loop has terminated.

```
i = 1
while i < 6:
  print(i)
  if i == 3:
    break
  i += 1
```

The output will be:

```
1
2
3
```

- A while loop that iterates as long as the value of i is less than 6.
- During each iteration of the loop, the current value of i is printed to the console.
- If the value of i is equal to 3, the break statement is executed, causing the loop to terminate prematurely.
- Therefore, the loop executes three times, printing the numbers 1, 2, and 3 to the console, and then exits.

```
i = 1
while i < 6:
  print(i)
  i+= 1
```

```
  if (i == 3):
    break
```

The output will be:

```
1
2
```

- A while loop that iterates as long as the value of i is less than 6.
- During each iteration of the loop, the current value of i is printed to the console.
- After printing the value of i, the value of i is incremented by 1.
- If the value of i becomes equal to 3, the break statement is executed, causing the loop to terminate prematurely.
- As a result, the loop executes twice, printing the numbers 1 and 2 to the console, and then exits.

```
1
2
3
4
5
```

```
i=1
while i<4:
    if i == 5:
        break
    print(i,end=" ")
    i = i + 1
else:
    print("Else is also printed")
```

The output will be:

```
1 2 3 Else is also printed
```

Note: The else block just after for/while is executed only when the loop is NOT terminated by a break statement.

In this example, the break statement will never hit because the value of “i” will not go above 4.

```
my_list = [10, 20, 30, 40, 50]
search_value = 30
for value in my_list:
    if value == search_value:
        print("Value found!")
        break
```

The output will be:

```
Value found!
```

- If the value is found within the list, it prints "Value found!" to the console and exits the loop using the break statement.
- As the value 30 is present in the list, the output will be "Value found!".

```
if True:
    break  # Error: 'break' outside loop
```

In this example,

- The if statement condition if True: is always true.
- However, the break statement is intended for use within loops to control their flow.
- Since there is no loop context surrounding the break statement, attempting to execute break outside of a loop results in a syntax error.
- Python will raise a SyntaxError with the message 'break' outside loop when attempting to run this code.

Note: The break statement is only allowed inside loops (for and while loops) and not within list comprehensions or generator expressions. The reason for this restriction is that list comprehensions and generator expressions are designed to create new sequences based on existing sequences, but they are not intended to have control flow statements like break, continue or pass.

```
# 'break' within a list comprehension
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
threshold = 5
result = [x if x <= threshold else break for x in numbers]
print(result)  # This will result in a syntax error
```

The output will be:

```
SyntaxError: invalid syntax
```

```
my_list = [1, 2, 3, 4, 5]
result = []
for x in my_list:
    if x == 3:
        break
    result.append(x)
print(result)
```

The output will be:

```
[1, 2]
```

In this example, the loop iterates over my_list, and if it encounters a value equal to 3, it breaks out of the loop. Otherwise, it appends each value to the result list.

3. **Nested Loops:** If a break statement is used within nested loops (loops within loops), it only exits the innermost loop where it is encountered. The outer loops continue their iterations unaffected.

```
#Using break statement in nested loops
for i in range(3):
    print("Outer loop iteration:", i)
    for j in range(3):
        print("Inner loop iteration:", j)
        if j == 1:
            print("Breaking inner loop...")
            break
    print("End of outer loop iteration:", i)
```

The output will be:

```
Outer loop iteration: 0
Inner loop iteration: 0
Inner loop iteration: 1
Breaking inner loop...
```

```
End of outer loop iteration: 0
Outer loop iteration: 1
Inner loop iteration: 0
Inner loop iteration: 1
Breaking inner loop...
End of outer loop iteration: 1
Outer loop iteration: 2
Inner loop iteration: 0
Inner loop iteration: 1
Breaking inner loop...
End of outer loop iteration: 2
```

In this example,

- We have an outer loop that iterates over the range from 0 to 2.
- Inside the outer loop, we have an inner loop that also iterates over the range from 0 to 2.
- Within the inner loop, we have a conditional statement that checks if j is equal to 1. If it is, the inner loop is broken using the break statement.
- When j reaches 1, the inner loop breaks, and the control goes back to the outer loop for the next iteration.
- This process repeats for each iteration of the outer loop.

(Prime numbers: A prime number is a whole number greater than 1 whose only factors are 1 and itself.)

```
# Iterate through numbers from 2 to 20
for num in range(-1, 21):
    if num <= 1:
        print(num, "is not a prime number")
    elif num > 1:
       # check for factors
       for i in range(2, num):
           if (num % i) == 0:
               print(num, "is not a prime number")
               break
       else:
           print(num, "is a prime number")
```

The output will be:

```
-1 is not a prime number
0 is not a prime number
```

```
1 is not a prime number
2 is a prime number
3 is a prime number
4 is not a prime number
5 is a prime number
6 is not a prime number
7 is a prime number
8 is not a prime number
9 is not a prime number
10 is not a prime number
11 is a prime number
12 is not a prime number
13 is a prime number
14 is not a prime number
15 is not a prime number
16 is not a prime number
17 is a prime number
18 is not a prime number
19 is a prime number
20 is not a prime number
```

- The loop iterates through the numbers from -1 to 20.
- For each number num: If num is equal to 1 or less than 1, it prints that num is not a prime number. This handles the cases where num is less than 2.
- If num is greater than 1, it proceeds to the next step to check if num is prime.
- For each number num greater than 1: It iterates through all the numbers from 2 to num - 1 (inclusive) to check if any of them are factors of num.
- If num is divisible by any number in this range (i.e., if num % i == 0), it means num is not prime, and it prints that num is not a prime number.
- If no factors are found in the range, it means that num is prime, and it prints that num is a prime number.

4. **Error Handling:** break can also be used for error handling or to handle exceptional cases where further iterations are not required.

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

for number in numbers:
    print(number)
    if number == 5:
        break
```

The output will be:

In this example, once the loop encounters the number 5, it executes the break statement. This means that further iterations through the rest of the list are not required because we've already found what we were looking for.

Understand the continue statement

Imagine you're still in the mall, visiting every store on a particular floor to find a gift for your friend. However, this time, you've set a different rule for yourself: if you find a store that doesn't interest you or doesn't have anything relevant, you'll skip that store and move on to the next one without wasting time browsing its merchandise.

In this, visiting each store still represents an iteration of a loop. The loop keeps running until you find the perfect gift or until you've visited all the stores on the floor.

The continue statement in Python is like encountering a store that doesn't interest you. When you encounter a specific condition (e.g., the store doesn't have anything relevant), the continue statement kicks in. It's like saying, "Okay, this store isn't worth my time. Let's skip it and move on to the next one."

So, when you execute the continue statement in your code, it's similar to skipping a store in the mall that doesn't meet your criteria. The continue statement skips the remaining statements in the current iteration of the loop and moves the control flow to the next iteration, without terminating the entire loop.
Syntax:

```
for item in sequence:
    if condition:
        continue
```

or

```
while condition:
    if another_condition:
        continue
```

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

for num in numbers:
    if num % 2 == 0:  # Check if the number is even
```

```
        continue
    print(num)   # This line will only be executed for odd numbers
```

The output will be:

```
1
3
5
7
9
```

In this example, the continue statement is used within an if condition that checks if the current number num is even. If the condition num % 2 == 0 is True (i.e., the number is even), the continue statement is executed, skipping the print(num) statement for that iteration, and the loop moves to the next iteration.

```
# Incorrect usage of continue statement outside of a loop
x = 10
while x > 0:
    if x % 2 == 0:
        print("Even number found:", x)
    else:
        continue  # This will raise a SyntaxError
    x -= 1
```

Executing this code will result in a SyntaxError because the continue statement is placed within a conditional block (else).

```
# Print only even numbers from 1 to 10 using a while loop
num = 1
while num <= 10:
    if num % 2 != 0:
        num += 1
        continue
    print(num)
    num += 1
```

The output will be:

```
2
```

```
4
6
8
10
```

- We initialize num to 1 and enter a while loop that continues as long as num is less than or equal to 10.
- Inside the loop, we check if num is odd (num % 2 != 0). If it is, we use continue to skip the rest of the code for the current iteration and move to the next iteration.
- If num is even, we print its value.
- After each iteration, we increment num by 1 to proceed to the next number.

```
i=1
while i<3:
    j=0
    while j<5:
        j = j +1
        if j==3:
            continue
        print(j, end = " ")
    i = i +1
```

The output will be:

```
1 2 4 5 1 2 4 5
```

In this example,

- The outer while loop runs as long as i is less than 3.
- Inside the outer loop, the inner while loop initializes j to 0 and runs as long as j is less than 5.
- Within the inner loop:
- j is incremented by 1 in each iteration.
- If j becomes equal to 3, the continue statement is executed, skipping the rest of the code block for that iteration and moving to the next iteration of the inner loop.
- If j is not equal to 3, the value of j is printed followed by a space.
- After the inner loop completes its iterations, i is incremented by 1 in each iteration of the outer loop.

Understand the pass statement

Imagine you're in a mall, and you've decided to visit every store on a particular floor to find a gift for your friend. However, as you're walking through the mall, you come across a store that is currently under renovation or temporarily closed. You can't enter the store or browse its merchandise at the moment.

In this analogy, visiting each store represents an iteration of a loop or a code block. The loop keeps running until you find the perfect gift or until you've visited all the stores on the floor.

The pass statement in Python is like encountering a store that is temporarily closed or under renovation. When you come across such a store, you can't do anything meaningful there at the moment, but you still need to acknowledge its existence and move on to the next store.

So, when you execute the pass statement in your code, it's similar to encountering a temporarily closed store in the mall. You can't execute any meaningful code or perform any actions there, but you need to have a placeholder or a dummy statement to keep your code syntactically valid.

With while loop:
Syntax:

```
while condition:
    pass
```

```
count = 0
while count < 5:
    pass  # Placeholder for future code
    print(count)
    count += 1
```

In this example, the while loop is intended to execute some logic while the count variable is less than 5. However, if you haven't yet implemented the logic that needs to go inside the loop, you can use the pass statement as a placeholder.

By including pass, you maintain the syntactic structure of the loop, ensuring that it remains valid Python code. This allows you to work on other parts of your program without causing syntax errors. Later on, you can replace pass with actual logic to be executed inside the loop.

The output will be:

```
0
1
2
```

```
3
4
```

With for loop:
Syntax:

```
for item in sequence:
    pass
    # Code block
```

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    pass  # Placeholder for future code
```

In this example, the pass statement is used within a for loop to iterate over a list of fruits. The loop iterates through each fruit in the fruits list, but the pass statement inside the loop serves as a placeholder, indicating that there is no code to execute at the moment.

This pattern is useful when you want to create a loop structure but haven't yet implemented the logic that needs to be executed within the loop.

With conditional statements (if, elif, else):
Syntax:

```
if condition:
    pass
    # Code block
elif another_condition:
    pass
    # Code block
else:
    pass
    # Code block
```

```
a = 33
b = 200

if b > a:
  pass
```

In this example, there's an if statement comparing the values of variables a and b. If b is greater than a, the pass statement is executed.

While this code doesn't perform any action when b is greater than a, it ensures that the syntactic structure of the if statement is maintained.

```
x = 5

if x > 0:
    pass  # Placeholder for future code
else:
    print("x is not positive")

# The program continues here after the if-else block
print("End of program")
```

The output will be:

```
End of program
```

Since x is indeed greater than 0, the if block is entered, but because of the pass statement, no specific action is performed. Therefore, the program directly moves to the next statement after the if-else block, printing "End of program" to the console.